

Iteración definida

Ciclos for y range() en Python

Programación I

Outline

- ¿Por qué repetir código?
- Iteración definida
- El ciclo for, paso a paso
- range() y sus 3 formas
- Ejemplos guiados
- Errores comunes
- Ejercicios

```
for i in range(5):  
    print(i)
```

¿Por qué necesitamos ciclos?

Imagina imprimir "Hola" 100 veces...

Sin ciclo

```
print("Hola")  
print("Hola")  
print("Hola")  
# ... 97 líneas más ...
```

Con ciclo

```
for i in range(100):  
    print("Hola")
```

2 líneas hacen el trabajo de 100.

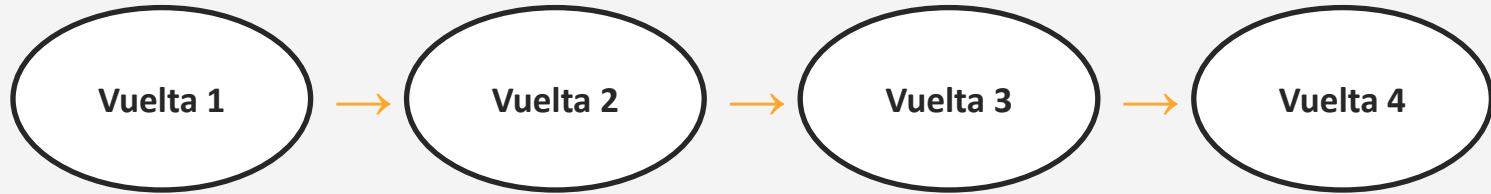
Control de flujo:

Ciclos / loops / bucles

Iteración: repetir n veces un proceso (o un *statement*)

Iteración definida: conocemos **con anticipación** cuántas veces se repetirá

Analogía: dar 4 vueltas a la pista 🏃



Sabes desde el inicio que serán 4 — eso es iteración definida.

Ciclo *For*

Iteración definida

Ciclo For — sintaxis

Syntax frecuente:

```
for <contador> in range(<inicio>, <fin>, <paso>):  
    <bloque que se repite>
```

Los **dos puntos (:)** y la **indentación (4 espacios)** son obligatorios.

Anatomía: cada parte explicada

```
for i in range(1, 6):
```

for

Palabra clave:
"repite esto"

i

El contador.
Cambia en cada vuelta

in range(1, 6)

La lista de valores
que tomará i

:

Abre el bloque
que se repetirá

El nombre i es costumbre (viene de index) — puede llamarse contador, numero, etc.

range() — forma 1 de 3

range(fin)

Con un solo número: empieza en 0 y termina uno antes de fin.

```
for i in range(5):  
    print(i)
```

OUTPUT

0

1

2

3

4

Valores que toma i:

0

1

2

3

4

5



El 5 NO se incluye



Empieza en 0, no en 1

range() — forma 2 de 3

range(inicio, fin)

Con dos números: empieza en inicio y termina uno antes de fin.

```
for i in range(1, 6):  
    print(i)
```

OUTPUT

1
2
3
4
5

Valores que toma i:

1

2

3

4

5

6



*El fin (6) nunca
se incluye*

range() — forma 3 de 3

range(inicio, fin, paso)

Con tres números: el paso indica de cuánto en cuánto avanza.

```
for i in range(0, 10, 2):  
    print(i)
```

OUTPUT

0
2
4
6
8

Valores que toma i:

0

2

4

6

8

10

*Avanza de 2 en 2.
Con paso negativo (-1)
¡cuenta hacia atrás!*

Regla de oro

El fin nunca se incluye

Si quieres llegar HASTA un número, suma 1 al fin:

Quiero contar 1 al 5...

```
range(1, 5) # 1,2,3,4
```

La solución: fin + 1

```
range(1, 6) # 1,2,3,4,5
```

¿Qué hace Python por dentro?

Seguimiento paso a paso

```
for i in range(1, 4):  
    print("Vuelta", i)
```

OUTPUT

```
Vuelta 1  
Vuelta 2  
Vuelta 3
```

Vuelta	Valor de i	¿Qué imprime?
1ª	i = 1	Vuelta 1
2ª	i = 2	Vuelta 2
3ª	i = 3	Vuelta 3
—	i = 4	Se detiene (4 = fin)

Truco para la clase: hagan esta tabla a mano antes de correr el código.

Herramienta clave

f-strings: texto + variables

```
nombre = "Ana"  
print(f"Hola, {nombre}!")
```

OUTPUT

Hola, Ana!

f

Indica que es un f-string (string formateado)

{ }

Las llaves insertan el valor de una variable en el texto

{i}

Dentro de un ciclo, se reemplaza por el valor actual de i

Ejemplo guiado #1

Feliz cumpleaños 🎂

Pedimos la edad y contamos hasta ella:

```
edad = int(input("¿Cuántos años cumples? "))

print("¡Feliz cumpleaños!")
for i in range(1, edad + 1):
    print(f"{i} 🎈")
```

OUTPUT

```
¿Cuántos años cumples? 3
¡Feliz cumpleaños!
1 🎈
2 🎈
3 🎈
```

`int(input(...))` convierte el texto a número · `edad + 1` para incluir la edad

Ejemplo guiado #2

Cuenta regresiva

Con paso negativo (-1), range cuenta hacia atrás:

```
n = 5

for i in range(n, -1, -1):
    print(i)

print("¡Despegue! )
```

OUTPUT

5

4

3

2

1

0

¡Despegue! 

¿Por qué -1 como fin? Para que el 0 sí se incluya (el fin se excluye, incluso al revés).

Ejemplo guiado #3

Múltiplos de 5 (dos caminos)

Camino A: usar el paso

```
for i in range(5, 51, 5):  
    print(i)
```

OUTPUT

```
5  
10  
15  
...  
50
```

Camino B: preguntar con %

```
for i in range(1, 51):  
    if i % 5 == 0:  
        print(i)
```

Mismo resultado, dos ideas distintas. El % (módulo) da el residuo de la división: si es 0, el número es múltiplo. Conecta el tema con los condicionales que ya vimos.

Patrón muy usado

El acumulador: sumar con un ciclo

```
total = 0
for i in range(1, 5):
    total = total + i

print(total)  # 10
```

i	total antes	total después
1	0	$0 + 1 = 1$
2	1	$1 + 2 = 3$
3	3	$3 + 3 = 6$
4	6	$6 + 4 = 10$

total empieza en 0 y va "acumulando" cada valor de i. Así se calculan sumas, promedios y totales.

¿Dudas / comentarios?

Gracias :)